Practical 4

```
import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score
)

# Load Dataset
df = pd.read_csv("/content/spam.csv", encoding='latin-1')

# Keep only required columns
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

# Convert labels
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Feature Extraction
vectorizer = TfidfVectorizer(stop_words='english')

X = vectorizer.fit_transform(df['message'])
y = df['label']

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# KNN Classifier
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

# Prediction
y_pred = knn.predict(X_test)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
```

```python
TN, FP, FN, TP = cm.ravel()

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
sensitivity = recall_score(y_test, y_pred)      # Recall
specificity = TN / (TN + FP)

# Results
print("\nConfusion Matrix")
print(cm)

print("\nPerformance Metrics")
print(f"Accuracy    : {accuracy:.4f}")
print(f"Precision   : {precision:.4f}")
print(f"Sensitivity : {sensitivity:.4f}")
print(f"Specificity : {specificity:.4f}")
```

```
Confusion Matrix
[[965   0]
 [100  50]]

Performance Metrics
Accuracy    : 0.9103
Precision   : 1.0000
Sensitivity : 0.3333
Specificity : 1.0000
```